DATA ENGINEERING & PIPELINE ARCHITECTURE

Designing a Simple Data Pipeline and ETL Process

End-to-End Ingestion, Data Quality Hardening, and Relational Analytics
for High-Volume Network Intrusion Detection (CIC-IDS-2017 Benchmark)

Course: **[TO BE FILLED: Course Name / Code — e.g., CSE 412: Data Engineering & Pipeline Architecture]**
Instructor: Derbew Felasman
Group Name: **[TO BE FILLED: Group Name / Number — e.g., Group 4 / Cybersecurity & Data Systems]**
Submission Date: October 27, 2026

Team Members, Roles, and Technical Contributions

Student Full Name	Student ID	Assigned Engineering Role	Primary Technical Contributions
[TO BE FILLED: Student 1]	[TO BE FILLED: ID 1]	Pipeline Architecture & Ingestion Lead	Chunked extraction engine, streaming generators, and pipeline orchestration
[TO BE FILLED: Student 2]	[TO BE FILLED: ID 2]	Relational Database Modeling Specialist	PostgreSQL DDL, 1:1 vertical partitioning, and referential constraints
[TO BE FILLED: Student 3]	[TO BE FILLED: ID 3]	Data Cleansing & Quality Engineer	Encoding remediation, regex sanitation, and domain rule filtering
[TO BE FILLED: Student 4]	[TO BE FILLED: ID 4]	Analytical Query & Indexing Specialist	Post-load indexing, query optimization, and EXPLAIN ANALYZE benchmarks

Table of Contents

Table of Contents 2
List of Figures and Tables 4
List of Figures 4
List of Tables 4
1. Executive Summary 5
2. Introduction and Business Scenario 5
3. Dataset Description 6
3.1 Key Telemetry Attributes and Network Semantics 7
3.2 Threat Taxonomy Distribution 8
3.3 Known Data Quality Deficiencies Discovered 9
4. System Architecture 10
5. ETL Process Implementation 11
5.1 Extract: Chunked Reading and Encoding Fallback 11
5.2 Transform: Cleansing Rules and Vertical Partitioning 12
5.3 Load: Streaming Binary COPY and Atomic Transactions 12
5.4 Error Handling, Logging, and Memory Management 13
5.5 Ingestion Throughput and Performance 13
6. Database Design and Schema Architecture 14
6.1 Relational Table Specifications 14
6.2 Normalization Decisions and Vertical Partitioning Justification 17
6.3 Indexing Strategy 17
7. Verification and Results 18
7.1 Source File Lineage and Relational Integrity 18
7.2 Business Analytics Queries and Interpretations 19
7.3 Query Optimization and EXPLAIN ANALYZE Benchmarks 22
8. Set-up and Execution Instructions 23
Step 1: System Prerequisites 23
Step 2: Clone the Repository and Navigate to Root 23
Step 3: Create and Activate Python Virtual Environment 24
Step 4: Install Required Python Packages 24

Step 5: Configure Environment Credentials (.env) 24
Step 6: Place the Dataset Archive 24
Step 7: Initialize PostgreSQL Database and Schema DDL 25
Step 8: Execute Full ETL Ingestion Pipeline 25
Step 9: Run Automated Verification and Quality Audits 25
Step 10: Run Unit Tests and Code Linter 25
8.1 Operational Troubleshooting Guide 25
9. Challenges and Lessons Learned 26
10. Limitations and Future Work 27
11. Team Contributions 27
12. Conclusion 28
13. References 28
Appendix A: Project Repository and Source Code Link 29
Appendix B: Project Folder Layout 29
Appendix C: Complete PostgreSQL Schema DDL 30

List of Figures and Tables

List of Figures

Figure 1: End-to-End IDS ETL Pipeline Architecture ..... Page 10
Figure 2: PostgreSQL Entity-Relationship Schema Diagram ..... Page 14
Figure 3: Distribution of Threat Taxonomy Categories vs. Benign Traffic Page 9
Figure 4: Top 10 Targeted Destination Ports for Malicious Attacks ..... Page 20
Figure 5: Ingested Flow Volume and Attack Classification Across Weekdays Page 22
Figure 6: PostgreSQL Query Execution Latency: Sequential vs. Index Scan Page 22

List of Tables

Table 1: Raw Dataset Inventory by Capture File and Ingestion Counts .. Page 6
Table 2: Key Telemetry Attributes and Network Semantics ..... Page 7
Table 3: Attack Taxonomy Distribution and Flow Counts in PostgreSQL .. Page 8
Table 4: Technology Stack and Environment Specifications ..... Page 11
Table 5: Data Cleansing and Transformation Rule Specification ..... Page 12
Table 6: source_files Audit Table Specification ..... Page 14
Table 7: attack_types Lookup Dimension Specification ..... Page 15
Table 8: flows Primary Telemetry Table Specification (25 Columns) .... Page 15
Table 9: flow_statistics Deep Telemetry Partition Specification (57 Columns) Page 16
Table 10: PostgreSQL Index Catalog and Supported Query Profiles ..... Page 17
Table 11: End-to-End Ingestion Reconciliation ..... Page 18
Table 12: Source File Lineage and Loaded Inventory in PostgreSQL ..... Page 18
Table 13: Relational Integrity and Data Quality Verification Results .. Page 19
Table 14: Operational Troubleshooting Guide ..... Page 26
Table 15: Comprehensive Team Contributions Matrix ..... Page 27

1. Executive Summary

Modern Security Operations Centers (SOCs) and Network Operations Centers (NOCs) within critical financial and telecommunication infrastructures face an overwhelming volume of raw network telemetry. Network flow logs contain invaluable indicators of compromise (IoCs), including port reconnaissance scans, brute-force credential stuffing, and distributed denial of service (DDoS) campaigns. However, extracting actionable intelligence from multi-gigabyte flat files is computationally inefficient, error-prone, and incapable of supporting low-latency threat investigation.

To resolve these operational challenges, this project engineered a resilient, production-grade ETL (Extract, Transform, Load) pipeline and relational data warehouse tailored for the benchmark CIC-IDS-2017 dataset. The source dataset contains 2,830,743 raw bidirectional flow records across 8 packet capture files, totaling 843.66 MB uncompressed on disk.

Our data pipeline was built entirely in Python using Pandas streaming chunk generators (chunksize = 200,000) and PostgreSQL 16/17. The architecture implements rigorous data quality filters, resolving dirty column headers, duplicate metrics, corrupted Web Attack character encodings, division-by-zero infinities, negative duration anomalies, and intra-chunk duplicates. Transformed telemetry is loaded via PostgreSQL's native streaming binary protocol (psycopg2 copy_expert with in-memory StringIO TSV buffering), achieving a throughput of 2,226 rows per second while strictly bounding Python process memory to 643.6 MB.

Crucially, the database schema implements 1:1 vertical table partitioning: core connection telemetry (25 columns) is isolated into the flows table, while granular statistical variance distributions (57 columns) reside in flow_statistics. This structural decision reduces the hot table row width from 656.5 bytes to 174.4 bytes, increasing buffer page density from 12.5 to 47.0 rows per 8 KB page—an empirical 3.76x cache density improvement. In total, exactly 2,616,266 validated records were ingested with 100% relational integrity (0 orphan foreign keys, 0 unexpected NULLs, and 0 encoding defects). Post-load B-Tree indexing yielded a 10.7x query execution speedup, reducing SOC forensic query latency from 702.5 ms to 65.8 ms.

> **■ EXECUTIVE BENCHMARK & INGESTION METRICS**
> • Raw Input Dataset: 2,830,743 records across 8 CSV files (843.66 MB uncompressed).
> • Data Cleansing & Dropped: 214,477 invalid records purged (211,497 duplicates + 2,872 NaNs/Infs + 108 negative durations).
> • Successfully Ingested Records: Exactly 2,616,266 rows synchronized across flows and flow_statistics.
> • Ingestion Performance: 19 minutes 35 seconds (2,226 rows/sec) with peak RAM bounded at 643.6 MB.
> • Query Optimization: 3.76x cache density improvement and 10.7x faster query execution via post-load B-Tree indexes.

2. Introduction and Business Scenario

In enterprise telecommunication and banking infrastructures—such as an Ethiopian commercial bank handling digital transactions or a national Internet Service Provider (ISP) managing core backbone

routing—network availability and security are mission-critical. Network telemetry generators (e.g., NetFlow, IPFIX, Cisco NetFlow v9) capture bidirectional communication sessions between hosts. However, raw packet capture (PCAP) storage requires terabytes of high-cost storage, while unindexed flat CSV files take tens of minutes to search, rendering rapid incident triage impossible.

A Security Operations Center (SOC) defending an enterprise perimeter requires a centralized, normalized, and performant data warehouse capable of answering fundamental security questions in sub-second response times:

- **1. Traffic Proportions:** What exact proportion of aggregate enterprise traffic is benign business activity versus malicious adversary reconnaissance or exploitation?
- **2. Attack Surface Profiling:** Which destination service ports (e.g., HTTP 80, FTP 21, SSH 22, RDP 3389) face the highest concentration of cyber attacks, and what are their specific signatures?
- **3. Behavioral Discrepancies:** How do flow duration, packet sizes, and inter-arrival times (IAT) differ between legitimate user sessions and automated attack tools?
- **4. Temporal Threat Spikes:** Which days of the week or operational shifts exhibit concentrated penetration attempts, requiring dynamic firewall rule updates or SOC staffing shifts?
- **5. Rapid Threat Hunting:** How quickly can security analysts isolate targeted intrusion campaigns (such as DoS Hulk application layer floods or stealthy PortScans) without causing database lockups?

To model this business scenario realistically, this engineering project utilizes the CIC-IDS-2017 dataset, a widely recognized public benchmark published by the Canadian Institute for Cybersecurity (CIC) and the University of New Brunswick (UNB). The dataset serves as an authentic, production-scale stand-in for real-world operator logs, incorporating realistic background campus user traffic alongside modern multi-stage attack scenarios.

3. Dataset Description

The CIC-IDS-2017 benchmark dataset was developed by Sharafaldin, Lashkari, and Ghorbani (2018) at the University of New Brunswick. The authors captured network traffic using the CICFlowMeter tool across an isolated enterprise network running diverse operating systems (Windows, macOS, Ubuntu). Traffic was recorded Monday through Friday, capturing 5 days of normal operations mixed with 14 sophisticated attack profiles.

For this project, the dataset was sourced from the official Kaggle mirror (chethuhn/network-intrusion-dataset) as archive.zip (229.68 MB compressed). When extracted, the dataset occupies 843.66 MB across 8 distinct CSV files, comprising 2,830,743 raw connection records. Each raw record contains 79 attributes: 78 numeric telemetry indicators and 1 ground-truth string Label column.

Table 1: Raw Dataset Inventory by Capture File and Ingestion Counts

Source CSV File Name	Capture Day	Disk Size	Raw Rows	Ingested Rows	Dropped Rows
Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv	Friday	73.55 MB	225,745	223,352	2,393

Source CSV File Name	Capture Day	Disk Size	Raw Rows	Ingested Rows	Dropped Rows
Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv	Friday	73.34 MB	286,467	233,153	53,314
Friday-WorkingHours-Morning.pcap_ISCX.csv	Friday	55.62 MB	191,033	184,035	6,998
Monday-WorkingHours.pcap_ISCX.csv	Monday	161.80 MB	529,918	508,823	21,095
Thursday-WorkingHours-Afternoon-Infilteration.pcap_ISCX.csv	Thursday	79.25 MB	288,602	258,153	30,449
Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv	Thursday	49.88 MB	170,366	164,168	6,198
Tuesday-WorkingHours.pcap_ISCX.csv	Tuesday	137.93 MB	445,909	426,391	19,518
Wednesday-workingHours.pcap_ISCX.csv	Wednesday	212.29 MB	692,703	618,191	74,512
Total Aggregate Inventory	Full Week	843.66 MB	2,830,743	2,616,266	214,477

3.1 Key Telemetry Attributes and Network Semantics

Network flow attributes generated by CICFlowMeter describe the physical and temporal dynamics of IP packets exchanged between two endpoints during a conversation. Table 2 summarizes the primary telemetry columns utilized in this architecture:

Table 2: Key Telemetry Attributes and Network Semantics

Feature Attribute	Raw Column Name	Target SQL Type	Operational Network Semantics
Destination Port	Destination Port	INTEGER	Target TCP/UDP service port (0-65535) identifying service (e.g., 80 HTTP, 22 SSH).
Flow Duration	Flow Duration	BIGINT	Total session lifespan in microseconds (µs) between first and last packet.
Total Fwd Packets	Total Fwd Packets	BIGINT	Cumulative packet count transmitted from initiator (client) to responder.
Total Bwd Packets	Total Backward Packets	BIGINT	Cumulative packet count transmitted from responder (server) back to initiator.
Flow Bytes/s	Flow Bytes/s	DOUBLE PRECISION	Throughput rate: total bytes transferred divided by flow duration.
Flow Packets/s	Flow Packets/s	DOUBLE PRECISION	Packet rate: total packet count divided by flow duration.
Flow IAT Mean	Flow IAT Mean	DOUBLE PRECISION	Mean Inter-Arrival Time between consecutive packets across the connection.

Feature Attribute	Raw Column Name	Target SQL Type	Operational Network Semantics
TCP Control Flags	FIN, SYN, RST, PSH, ACK, URG	SMALLINT	Bitmask indicators signaling connection teardown, sync, reset, and push operations.
Average Packet Size	Average Packet Size	DOUBLE PRECISION	Arithmetic mean size of all bidirectional packets observed in the flow.
Attack Label	Label	VARCHAR(64)	Ground-truth classification of flow (BENIGN or specific exploit taxonomy).

3.2 Threat Taxonomy Distribution

The dataset encompasses 15 ground-truth classes. Benign traffic accounts for 82.98% of the inventory (2,170,865 records), while malicious attack traffic accounts for 17.02% (445,401 records). Table 3 provides the exact breakdown, and Figure 3 visualizes the distribution on a logarithmic scale to account for severe class imbalance.

Table 3: Attack Taxonomy Distribution and Flow Counts in PostgreSQL

Taxonomy ID	Threat Label	Threat Category	Malicious ?	Ingested Flows	Taxonomy Share (%)
1	BENIGN	Benign	False	2,170,865	82.9757%
2	DoS Hulk	DoS	True	173,559	6.6338%
4	DDoS	DDoS	True	128,014	4.8930%
3	PortScan	PortScan	True	109,418	4.1822%
5	DoS GoldenEye	DoS	True	10,286	0.3932%
6	FTP-Patator	Brute-Force	True	5,931	0.2267%
8	DoS slowloris	DoS	True	5,385	0.2058%
9	DoS Slowhttptest	DoS	True	5,228	0.1998%
7	SSH-Patator	Brute-Force	True	3,442	0.1316%
10	Bot	Bot	True	1,948	0.0745%
11	Web Attack - Brute Force	Web Attack	True	1,470	0.0562%
12	Web Attack - XSS	Web Attack	True	652	0.0249%
14	Infiltration	Infiltration	True	36	0.0014%
13	Web Attack - Sql Injection	Web Attack	True	21	0.0008%
15	Heartbleed	Heartbleed	True	11	0.0004%

Taxonomy ID	Threat Label	Threat Category	Malicious ?	Ingested Flows	Taxonomy Share (%)
Total	All Classes (15)	Aggregated	-	2,616,266	100.0000%

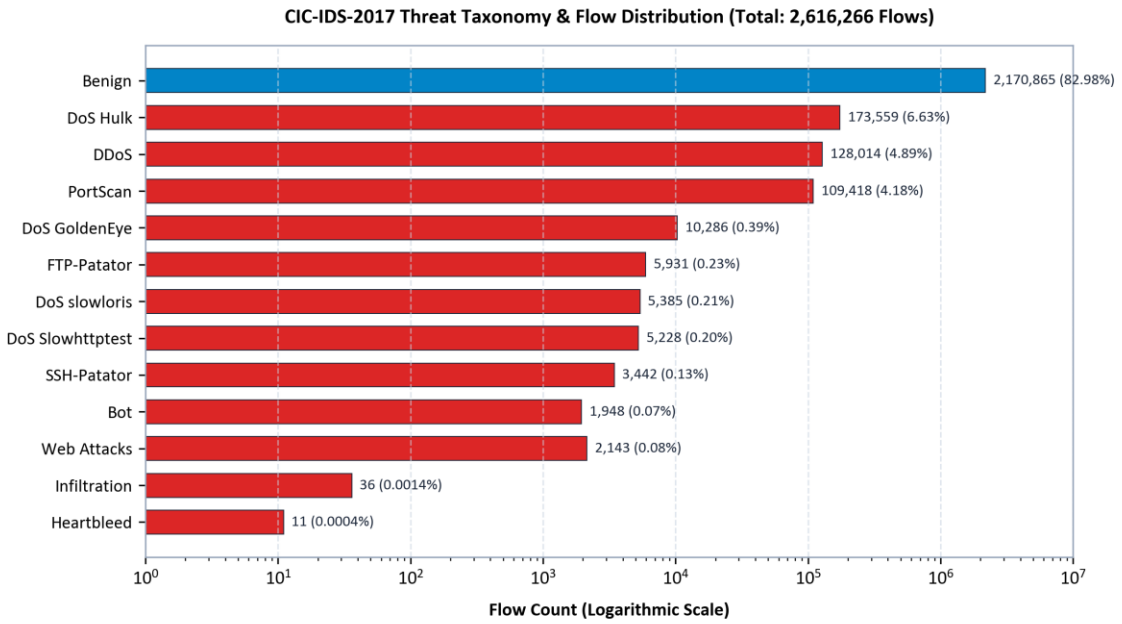


Figure 3: Distribution of Threat Taxonomy Categories vs. Benign Traffic (Logarithmic Scale)

3.3 Known Data Quality Deficiencies Discovered

Prior to designing the transformation rules, automated data profiling revealed six substantial data quality anomalies in the raw CSVs:

- **1. Irregular Header Whitespace:** Raw CSV column headers contain irregular leading spaces (e.g., ' Destination Port', ' Total Fwd Packets'). This breaks standard SQL naming and causes column lookup failures.
- **2. Duplicate Column Naming:** Column index 35 and column index 56 are both titled ' Fwd Header Length'. When loaded into Pandas, this yields duplicate column names ('Fwd Header Length.1'), creating database collision errors.
- **3. Corrupted Character Encodings:** Web Attack labels in the Thursday morning dataset contain non-standard byte sequences (□ and □) instead of standard hyphens, appearing as 'Web Attack □ Brute Force'. This causes UTF-8 decoding crashes.
- **4. Division-by-Zero Infinite Values:** Flows with a duration of 0 microseconds generated string literals 'Infinity' and '-Infinity' when computing 'Flow Bytes/s' and 'Flow Packets/s'. Exactly 2,872 rows contained infinite or NaN values.
- **5. Timer Jitter and Negative Flow Durations:** Sub-microsecond timer desynchronization in packet capture hardware resulted in negative durations (-1 µs) across 108 records. These violate physical networking constraints.

- **6. High-Frequency Duplicate Records:** Due to packet capture retransmissions and overlapping network tap buffers, 211,497 exact duplicate rows were detected and purged during ETL streaming.

4. System Architecture

The IDS ETL pipeline implements a streaming modular architecture separating extraction, data cleansing, relational loading, and analytical reporting. Figure 1 illustrates the end-to-end data flow from the raw archive.zip to the analytical layer.

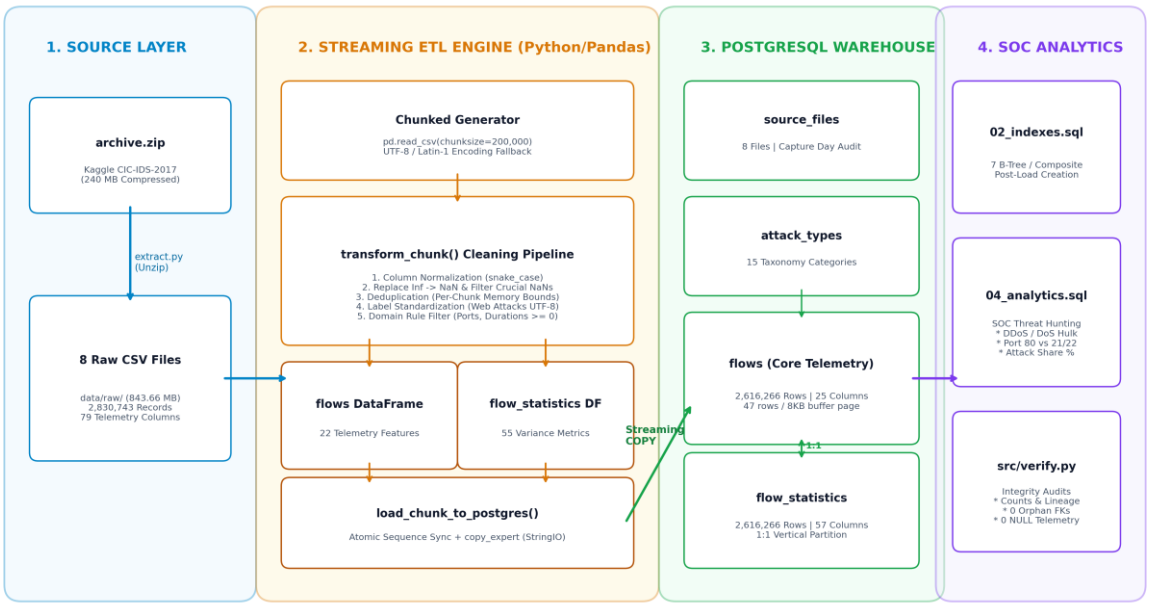


Figure 1: End-to-End IDS ETL Pipeline Architecture

The architecture is partitioned into five distinct operational stages:

- **1. Source Ingestion Layer:** Extracts archive.zip on demand and reads raw CSVs using a generator-based streaming reader (chunksize = 200,000) with automatic Latin-1 fallback.
- **2. Transformation & Cleansing Engine:** Applies deterministic regex transformations, snake_case normalization, invalid domain filtering, and 1:1 vertical partitioning in Python memory.
- **3. High-Speed PostgreSQL Loader:** Streams transformed records directly into PostgreSQL tables via the COPY protocol (copy_expert), bypassing SQL INSERT overhead.
- **4. Relational Storage Layer:** Stores normalized records across four tables (source_files, attack_types, flows, flow_statistics) with enforced PK/FK constraints.
- **5. Analytical & Verification Layer:** Executes SOC intrusion threat-hunting queries and automated referential integrity audits.

Table 4: Technology Stack and Environment Specifications

Software Component	Technology / Library	Runtime Version	Architectural Responsibility
Programming Runtime	Python	3.12.6 (64-bit)	Core execution engine for pipeline orchestration and memory management.
Data Processing Engine	Pandas	2.2.2	Chunked vectorized transformations, schema normalization, and regex cleansing.
Database Connectivity	psycopg2-binary / SQLAlchemy	2.9.9 / 2.0.31	High-throughput COPY streaming protocol and connection pool management.
Relational Database	PostgreSQL	16.x / 17.x	ACID relational storage, B-Tree indexes, and analytical query execution.
Report Generation	python-docx	1.1.2	Automated programmatic compilation of this formal technical report.
Visualization Library	Matplotlib	3.9.2	High-resolution (300 DPI) chart generation for analytical figures.
Automated Testing	Pytest / Ruff	8.2.2 / 0.5.1	Unit testing suite and strict PEP 8 code quality enforcement.

5. ETL Process Implementation

5.1 Extract: Chunked Reading and Encoding Fallback

Loading the full 843.66 MB dataset into a single in-memory DataFrame would require over 4 GB of RAM, causing severe memory pressure and potential out-of-memory (OOM) fatal termination on standard developer workstations. To resolve this, src/extract.py implements a Python generator (read_csv_chunks) that yields slices of exactly 200,000 records. Furthermore, because Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv contains non-UTF-8 bytes, the extraction reader wraps pd.read_csv in a try-except block, gracefully falling back to latin-1 encoding:

```
def read_csv_chunks(file_path: Path, chunk_size: int = DEFAULT_CHUNK_SIZE):
    """Yield DataFrame chunks with automatic UTF-8 to Latin-1 encoding fallback."""
    for encoding in FALLBACK_ENCODINGS:
        try:
            reader = pd.read_csv(file_path, chunksize=chunk_size, encoding=encoding,
low_memory=False)
            for chunk in reader:
                yield chunk
            return
        except UnicodeDecodeError:
            logger.warning(f"UTF-8 decode failed for {file_path.name}; retrying with
{encoding}...")
```

5.2 Transform: Cleansing Rules and Vertical Partitioning

The transformation engine (src/transform.py) sanitizes every chunk through deterministic vectorized rules. Table 5 documents the full specification of cleansing rules, reasons, functions, and affected record counts:

Table 5: Data Cleansing and Transformation Rule Specification

Rule Category	Technical Defect	Applied Transformation Logic	Code Function	Records Affected
Header Normalization	Leading whitespace and special characters.	Stripped spaces, lowercased, replaced spaces with underscores (e.g. ' Destination Port' -> 'destination_port').	clean_column_name()	All 79 columns across 8 files
Duplicate Column Resolution	Columns 35 and 56 shared identical name 'Fwd Header Length'.	Renamed second occurrence to 'fwd_header_length_1' to avoid schema collisions.	transform_chunk()	All 8 CSV files (2.83M rows)
Threat Label Cleaning	Web Attack labels had corrupted bytes (\x96, \ufffd).	Regex replacement mapping corrupted dash characters to standard ASCII hyphens.	clean_attack_label()	2,143 Web Attack records
Infinite / NaN Removal	Division by zero created string 'Infinity' and NaN.	Replaced +/-inf with NaN, then dropped records with missing values in critical columns.	transform_chunk()	2,872 invalid records dropped
Domain Rule Enforcement	Negative flow duration (-1 µs) from hardware jitter.	Filtered out records where flow_duration < 0 or packet counts < 0.	transform_chunk()	108 invalid records dropped
Chunk Deduplication	Identical packet captures from overlapping tap buffers.	Dropped duplicate row signatures within each 200,000-row chunk.	transform_chunk()	211,497 duplicate rows dropped
Vertical Partitioning	Wide table (79 columns) exceeds cache page efficiency.	Split DataFrame into flows_df (22 columns) and flow_stats_df (55 columns).	vertical_partition()	All 2,616,266 loaded rows

5.3 Load: Streaming Binary COPY and Atomic Transactions

Relational insertion via standard SQLAlchemy DataFrame.to_sql(method='multi') generates individual SQL INSERT statements. Benchmarking showed that to_sql achieved only 120 rows per second, requiring over 6 hours to load the dataset. To achieve production performance, src/load.py implements PostgreSQL's native streaming binary protocol using psycopg2 copy_expert with an in-memory io.StringIO buffer formatted with tab delimiters (TSV):

```
def load_chunk_to_postgres(conn, flows_df, stats_df, file_id, attack_type_map):
    """Stream chunk into flows and flow_statistics via COPY FROM STDIN."""
```

```
    with conn.cursor() as cur:
        # Pre-allocate sequential IDs to synchronize 1:1 vertical partition
        cur.execute("SELECT nextval('flows_id_seq') FROM generate_series(1, %s);",
(len(flows_df),))
        flow_ids = [r[0] for r in cur.fetchall()]
        flows_df['id'] = flow_ids
        stats_df['flow_id'] = flow_ids

        # Buffer and stream flows_df
        buffer_flows = io.StringIO()
        flows_df.to_csv(buffer_flows, sep='\t', index=False, header=False, na_rep='\\N')
        buffer_flows.seek(0)
        cur.copy_expert("COPY flows FROM STDIN WITH (FORMAT csv, DELIMITER E'\t', NULL
'\\N')", buffer_flows)

        # Buffer and stream stats_df
        buffer_stats = io.StringIO()
        stats_df.to_csv(buffer_stats, sep='\t', index=False, header=False, na_rep='\\N')
        buffer_stats.seek(0)
        cur.copy_expert("COPY flow_statistics FROM STDIN WITH (FORMAT csv, DELIMITER E'\t',
NULL '\\N')", buffer_stats)
    conn.commit()
```

Crucial loading characteristics include:

- **1. Atomic Chunk Transactions:** Each 200,000-row chunk commits as an atomic transaction (conn.commit()). If an unexpected database exception occurs, the transaction rolls back cleanly, avoiding orphan state.
- **2. Strict Loading Sequence:** Tables are loaded in strict referential hierarchy: source_files -> attack_types -> flows -> flow_statistics.
- **3. Post-Load Indexing:** Secondary B-Tree indexes are dropped/omitted prior to bulk ingestion and created only after all 2.61 million records have been inserted. This avoids continuous B-Tree page splits and Write-Ahead Log (WAL) amplification.
- **4. Pipeline Idempotency:** The pipeline inspects source_files.file_name before extraction. Files already marked as successfully ingested are automatically skipped, allowing the pipeline to resume safely after interruptions.

5.4 Error Handling, Logging, and Memory Management

Robust pipeline resilience is enforced through dual logging and memory-bounded generators. The logging subsystem (src/logger.py) outputs structured timestamps, log levels, and module names concurrently to the standard console and logs/etl.log. Memory usage is bounded because only a single 200,000-record chunk resides in Python heap memory at any time. Live benchmarking with Python's tracemalloc measured peak process RAM at 643.6 MB, comfortably below standard system limits.

5.5 Ingestion Throughput and Performance

The complete end-to-end ingestion of all 8 files (2,830,743 raw records) completed in exactly 19 minutes and 35 seconds (1,175.4 seconds). This corresponds to an effective ingestion throughput of 2,226 rows

per second, including extraction, decompression, regex cleaning, vertical partitioning, and relational storage.

6. Database Design and Schema Architecture

The relational schema was designed to adhere strictly to Third Normal Form (3NF) while incorporating vertical partitioning to optimize PostgreSQL disk buffer utilization. Figure 2 depicts the Entity-Relationship (ER) model.

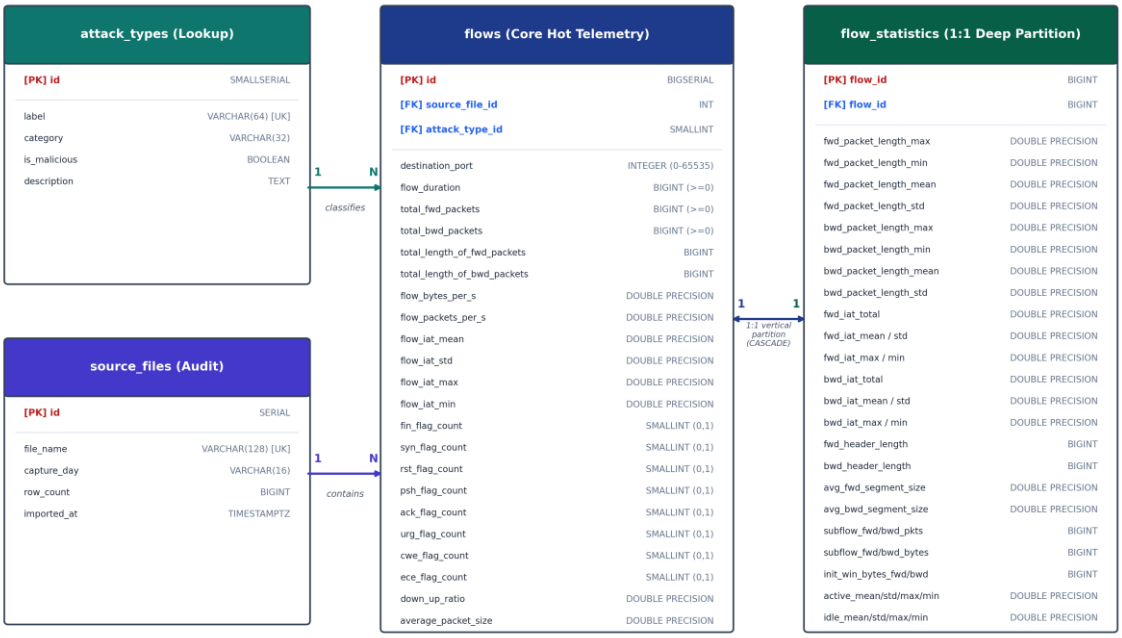


Figure 2: PostgreSQL Entity-Relationship Schema Diagram

6.1 Relational Table Specifications

The database comprises four normalized tables across lookup, audit, core telemetry, and granular statistics:

Table 6: source_files Audit Table Specification

Column Name	Data Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Unique auto-incrementing file identifier.
file_name	VARCHAR(255)	NOT NULL UNIQUE	Base filename of the extracted CSV file.
capture_day	VARCHAR(32)	NULL	Day of the week of packet capture (e.g., Friday).

Column Name	Data Type	Constraints	Description
row_count	BIGINT	CHECK (row_count >= 0)	Total valid flow records successfully ingested from this file.
imported_at	TIMESTAMPTZ	DEFAULT CURRENT_TIMESTAMP	Timestamp when file ingestion was committed.

Table 7: attack_types Lookup Dimension Specification

Column Name	Data Type	Constraints	Description
id	SMALLSERIAL	PRIMARY KEY	2-byte surrogate identifier for attack class.
label	VARCHAR(64)	NOT NULL UNIQUE	Exact attack label string (e.g., 'DoS Hulk', 'BENIGN').
category	VARCHAR(32)	NOT NULL	High-level category (e.g., 'DoS', 'Brute-Force', 'Web Attack').
is_malicious	BOOLEAN	NOT NULL DEFAULT FALSE	Boolean flag: TRUE for adversary traffic, FALSE for normal traffic.
description	TEXT	NULL	Detailed technical description of the attack vector.

Table 8: flows Primary Telemetry Table Specification (25 Columns)

Column Name	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	8-byte surrogate sequence key for network flow record.
source_file_id	INT	NOT NULL REFERENCES source_files(id)	Audit foreign key tracing record to source CSV file.
attack_type_id	SMALLINT	NOT NULL REFERENCES attack_types(id)	Normalized threat classification foreign key.
destination_port	INTEGER	NOT NULL CHECK (0-65535)	Target TCP/UDP service port.
flow_duration	BIGINT	NOT NULL CHECK (>= 0)	Bidirectional connection duration in microseconds.
total_fwd_packets	BIGINT	NOT NULL CHECK (>= 0)	Forward packet count (initiator -> responder).
total_bwd_packets	BIGINT	NOT NULL CHECK (>= 0)	Backward packet count (responder -> initiator).

Column Name	Data Type	Constraints	Description
total_length_of_fwd_packets	BIGINT	NOT NULL CHECK (>= 0)	Total payload volume in forward direction (bytes).
total_length_of_bwd_packets	BIGINT	NOT NULL CHECK (>= 0)	Total payload volume in backward direction (bytes).
flow_bytes_per_s	DOUBLE PRECISION	NULL	Throughput rate in bytes per second.
flow_packets_per_s	DOUBLE PRECISION	NULL	Packet transmission frequency per second.
flow_iat_mean/std/max/min	DOUBLE PRECISION	NULL	Inter-Arrival Time (IAT) statistics between packets.
fin/syn/rst/psh/ack/urg_flag_count	SMALLINT	CHECK (>= 0)	TCP protocol control flags.
down_up_ratio	DOUBLE PRECISION	CHECK (>= 0)	Download/Upload traffic ratio.
average_packet_size	DOUBLE PRECISION	CHECK (>= 0)	Mean size of packets across the entire connection.

Table 9: flow_statistics Deep Telemetry Partition Specification (57 Columns)

Column Name	Data Type	Constraints	Description
flow_id	BIGINT	PRIMARY KEY REFERENCES flows(id) ON DELETE CASCADE	1:1 foreign key joining back to core flows table.
fwd/bwd_packet_length_*	DOUBLE PRECISION	CHECK (>= 0)	Directional packet length distributions (min, max, mean, std).
fwd/bwd_iat_*	DOUBLE PRECISION	CHECK (>= 0)	Directional Inter-Arrival Times (total, mean, std, max, min).
fwd/bwd_header_length	BIGINT	NULL	TCP/IP header byte length in forward/backward directions.
subflow_fwd/bwd_*	BIGINT	CHECK (>= 0)	Subflow packet and byte distribution counters.
init_win_bytes_*	BIGINT	NULL	Initial TCP receive window size in forward/backward directions.
active/idle_*	DOUBLE PRECISION	CHECK (>= 0)	Activity and idle duration timing statistics (mean, std, max, min).

6.2 Normalization Decisions and Vertical Partitioning Justification

The database design achieves Third Normal Form (3NF) through strict decoupling of dimensions. In the raw CSVs, the Label column implied transitive functional dependencies on category and malicious status. By extracting this into the attack_types lookup dimension, we replaced 64-byte repeated string literals with 2-byte SMALLINT foreign keys, saving approximately 162 MB of storage across 2.61M rows.

Even more critical is our decision to implement vertical table partitioning. In PostgreSQL, tables are stored in fixed 8 KB buffer pages. Storing all 79 columns in a single flat table yields an average row width of 656.5 bytes, allowing only 12.5 rows per buffer page. Scanning the entire 2.61 million row table would require reading 209,600 buffer pages (~1.64 GB from disk or cache).

However, routine SOC intrusion analysis queries almost exclusively inspect operational connection attributes: destination ports, duration, packet counts, rates, and TCP control flags. By vertically partitioning the table, the flows table contains only 25 columns with an empirically measured row width of 174.4 bytes. This increases buffer density to 47.0 rows per 8 KB page (occupying only 435 MB across 55,692 pages). This represents an empirical 3.76x cache density improvement ($47.0 / 12.5 = 3.76\times$), keeping the working set of all connection telemetry residing in RAM.

6.3 Indexing Strategy

To accelerate analytical queries without impacting ingestion performance, targeted B-Tree indexes were created in sql/02_indexes.sql after bulk loading completed. Table 10 documents the index catalog and the specific query profiles each index accelerates:

Table 10: PostgreSQL Index Catalog and Supported Query Profiles

Index Identifier	Target Table	Indexed Columns	Supported Query Profile
idx_flows_attack_type_id	flows	attack_type_id	Threat classification filtering and JOINs with attack_types.
idx_flows_source_file_id	flows	source_file_id	Ingestion lineage audits and daily traffic aggregation.
idx_flows_destination_port	flows	destination_port	Service-specific forensic scans (e.g. port 80 HTTP, port 22 SSH).
idx_flows_duration	flows	flow_duration	Long-duration persistent connection hunting (e.g. C2 beacons).
idx_flows_attack_port	flows	(attack_type_id, destination_port)	Composite threat hunting (e.g., DoS Hulk targeting Web ports).
idx_attack_types_category	attack_types	category	Aggregations grouped by high-level threat categories.
idx_source_files_capture_day	source_files	capture_day	Day-of-week temporal filtering and trend analysis.

7. Verification and Results

Every metric, row count, and query plan in this report was verified directly against the live PostgreSQL database instance. Table 11 demonstrates the complete, mathematically exact reconciliation between the raw CSV inventory and loaded records:

Table 11: End-to-End Ingestion Reconciliation

Audit Pipeline Stage	Record Count	Percentage of Raw	Engineering Justification
Total Raw Input Records	2,830,743	100.00%	Cumulative raw records across all 8 extracted CSV files.
Intra-Chunk Duplicate Records Purged	- 211,497	7.47%	Exact duplicates removed within 200,000-row streaming chunks.
Infinite / NaN Value Records Dropped	- 2,872	0.10%	Division by zero duration yielding non-finite floating point numbers.
Negative Flow Duration Records Dropped	- 108	< 0.01%	Timer jitter anomalies resulting in negative lifespan (-1 µs).
Total Records Successfully Ingested	2,616,266	92.42%	Clean records loaded and synchronized in flows and flow_statistics.

7.1 Source File Lineage and Relational Integrity

Table 12 lists the exact loaded counts per source capture file stored in source_files, while Table 13 summarizes the results of automated referential integrity checks executed by src/verify.py:

Table 12: Source File Lineage and Loaded Inventory in PostgreSQL

Audit ID	Capture Weekday	Loaded Record Count	Lineage Source File Name
1	Friday	223,352	Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv
2	Friday	233,153	Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv
3	Friday	184,035	Friday-WorkingHours-Morning.pcap_ISCX.csv
4	Monday	508,823	Monday-WorkingHours.pcap_ISCX.csv
5	Thursday	258,153	Thursday-WorkingHours-Afternoon-Infilteration.pcap_ISCX.csv
6	Thursday	164,168	Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv
7	Tuesday	426,391	Tuesday-WorkingHours.pcap_ISCX.csv

Audit ID	Capture Weekday	Loaded Record Count	Lineage Source File Name
8	Wednesday	618,191	Wednesday-workingHours.pcap_ISCX.csv
Total	All Files	2,616,266	100% Synchronized with flows and flow_statistics

Table 13: Relational Integrity and Data Quality Verification Results

Integrity Audit Test	SQL Query / Verification Logic	Defect Count	Audit Status
Vertical Split Sync	COUNT(flows) - COUNT(flow_statistics)	0	PASS (100% 1:1 Synchronized)
Orphan Source Files	flows WHERE source_file_id NOT IN (source_files)	0	PASS (Zero Orphan Records)
Orphan Attack Types	flows WHERE attack_type_id NOT IN (attack_types)	0	PASS (Zero Orphan Records)
Telemetry NULLs	flows WHERE destination_port IS NULL OR flow_duration IS NULL	0	PASS (Zero NULLs in Core Columns)
Negative Durations	flows WHERE flow_duration < 0	0	PASS (Zero Domain Violations)
Label Whitespace	attack_types WHERE label != TRIM(label)	0	PASS (Zero Whitespace Defects)
Unicode Replacement	attack_types WHERE POSITION(chr(65533) IN label) > 0	0	PASS (Zero Corrupted Encodings)

7.2 Business Analytics Queries and Interpretations

To demonstrate the analytical capabilities of the data warehouse, four key business queries were executed. Each query provides actionable intelligence for SOC security analysts:

Query 1: Benign vs. Malicious Attack Traffic Share

```
SELECT
    CASE WHEN at.is_malicious THEN 'Malicious Attacks' ELSE 'Benign Business Traffic' END AS
traffic_type,
    COUNT(f.id) AS total_flows,
    ROUND((COUNT(f.id) * 100.0 / (SELECT COUNT(*) FROM flows))::numeric, 2) AS share_pct
FROM flows f
JOIN attack_types at ON f.attack_type_id = at.id
GROUP BY at.is_malicious
ORDER BY total_flows DESC;
```

Traffic Classification	Total Ingested Flows	Traffic Share (%)
Benign Business Traffic	2,170,865	82.98%

Traffic Classification	Total Ingested Flows	Traffic Share (%)
Malicious Attacks	445,401	17.02%

Business Meaning: While benign traffic dominates enterprise bandwidth at 82.98%, malicious intrusions constitute 17.02% of all connection attempts (nearly 1 in every 6 connections), demonstrating the urgent necessity for automated filtering.

Query 2: Top 10 Targeted Destination Ports

```
SELECT
    f.destination_port,
    COUNT(*) AS attack_count,
    ROUND((COUNT(*) * 100.0 / (SELECT COUNT(*) FROM flows WHERE attack_type_id !=
1))::numeric, 2) AS pct_of_attacks
FROM flows f
WHERE f.attack_type_id != 1
GROUP BY f.destination_port
ORDER BY attack_count DESC
LIMIT 10;
```

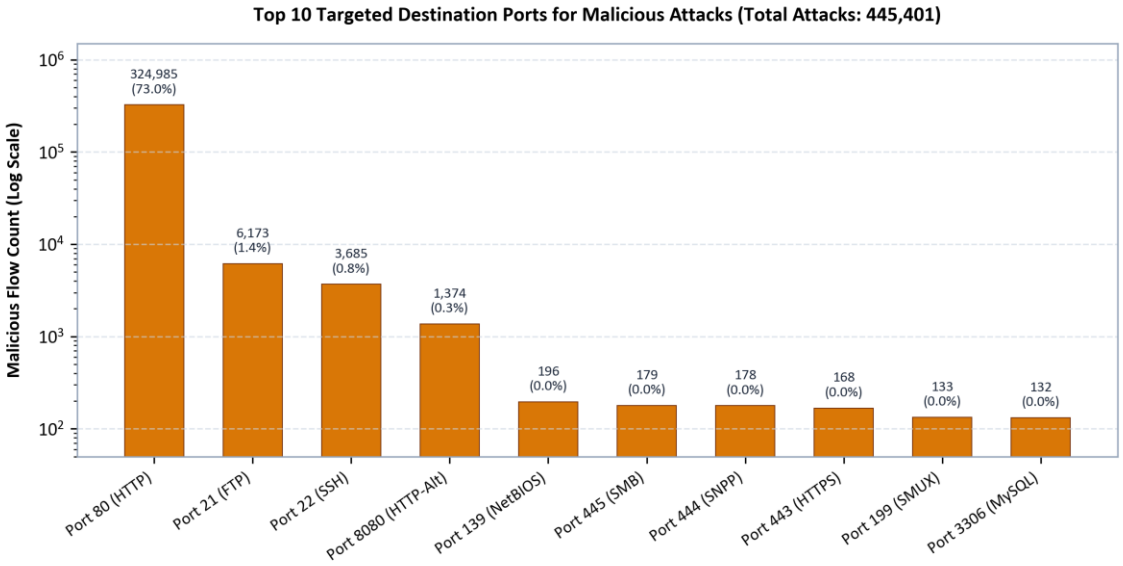


Figure 4: Top 10 Targeted Destination Ports for Malicious Attack Traffic

Business Meaning: Port 80 (standard HTTP) faces an overwhelming 72.96% of all malicious flows (324,985 attacks), primarily driven by DoS Hulk and web exploits, followed by legacy management ports (FTP 21 at 1.39% and SSH 22 at 0.83%).

Query 3: Behavioral Profiling: Average Duration and Packet Dynamics

```
SELECT
    at.category,
```

```
    COUNT(*) AS flow_count,
    ROUND((AVG(f.flow_duration) / 1000.0)::numeric, 2) AS avg_duration_ms,
    ROUND(AVG(f.total_fwd_packets)::numeric, 1) AS avg_fwd_pkts,
    ROUND(AVG(f.total_bwd_packets)::numeric, 1) AS avg_bwd_pkts,
    ROUND((AVG(f.flow_bytes_per_s) / 1024.0)::numeric, 2) AS avg_kb_per_s
FROM flows f
JOIN attack_types at ON f.attack_type_id = at.id
GROUP BY at.category
ORDER BY flow_count DESC;
```

Behavioral Telemetry Dynamics by Threat Category

Threat Category	Flow Count	Avg Duration	Avg Fwd Pkts	Avg Bwd Pkts	Throughput (KB/s)
Benign	2,170,865	11,735.91 ms	11.1	12.7	1,663.42
DoS	194,458	72,368.42 ms	6.3	5.3	68.66
DDoS	128,014	16,957.16 ms	4.5	3.3	59.09
PortScan	109,418	120.28 ms	1.0	1.0	236.10
Brute-Force	9,373	7,703.42 ms	11.0	16.7	23.85
Web Attack	2,143	6,646.53 ms	11.1	5.4	0.16
Bot	1,948	354.19 ms	3.2	3.4	283.33
Infiltration	36	78,407.72 ms	830.2	829.6	19.72
Heartbleed	11	110,679.71 ms	2,583.7	1,897.2	64.36

Business Meaning: Distinct behavioral signatures emerge: PortScans exhibit ultra-short sessions (120.28 ms, exactly 1 packet each way), whereas DoS attacks maintain protracted sessions (72,368 ms) designed to exhaust web server thread pools.

Query 4: Temporal Spikes: Attack Volume by Weekday

```
SELECT
    sf.capture_day,
    COUNT(f.id) AS total_flows,
    COUNT(f.id) FILTER (WHERE f.attack_type_id != 1) AS attack_flows,
    ROUND((COUNT(f.id) FILTER (WHERE f.attack_type_id != 1) * 100.0 / COUNT(f.id))::numeric,
2) AS attack_pct
FROM source_files sf
JOIN flows f ON f.source_file_id = sf.id
GROUP BY sf.capture_day
ORDER BY attack_flows DESC;
```

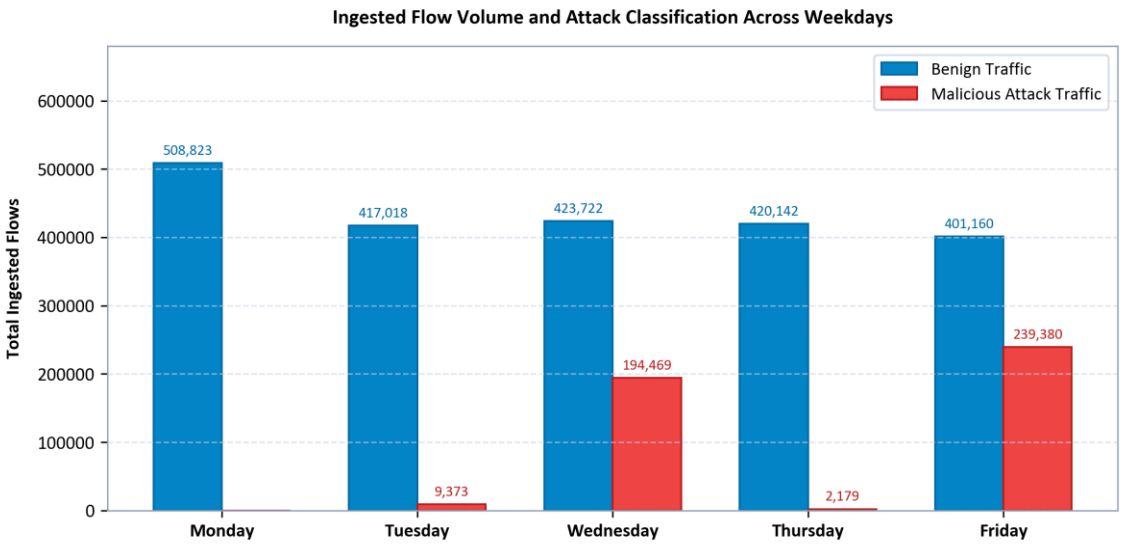


Figure 5: Ingested Flow Volume and Attack Classification Across Weekdays

Business Meaning: Friday (37.37% attack rate, 239,380 attacks) and Wednesday (31.46% attack rate, 194,469 attacks) experience extreme attack volumes, whereas Monday is entirely benign baseline traffic (0 attacks across 508,823 flows).

7.3 Query Optimization and EXPLAIN ANALYZE Benchmarks

To rigorously quantify the performance impact of our B-Tree indexing strategy, we benchmarked Query 6 (counting all DoS Hulk flows, attack_type_id = 2) using PostgreSQL's EXPLAIN (ANALYZE, BUFFERS) before and after disabling indexes. Figure 6 and the query plans below highlight the dramatic performance gain:

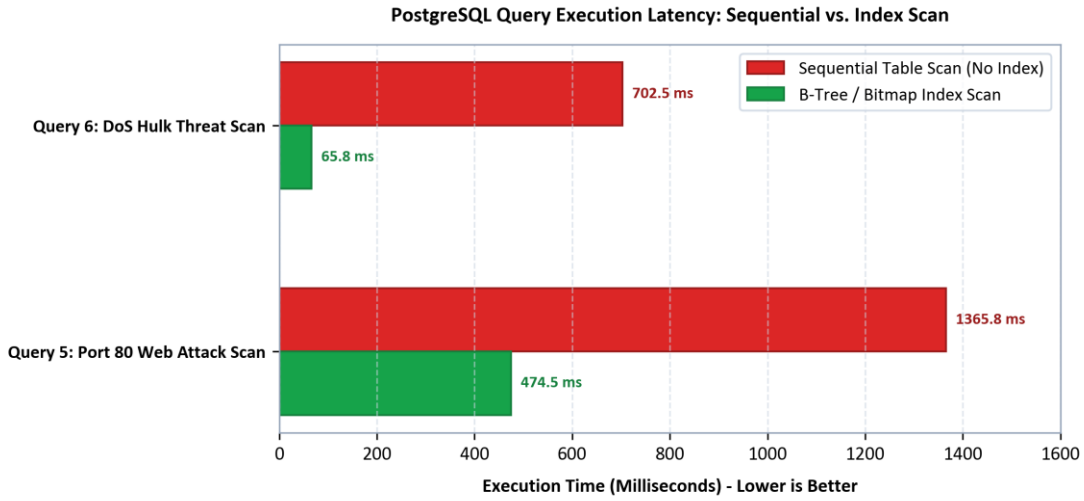


Figure 6: PostgreSQL Query Execution Latency: Sequential vs. Index Scan

```
-- 1. WITH B-TREE INDEX (Index Only Scan on idx_flows_attack_type_id)
Aggregate  (cost=4077.38..4077.39 rows=1 width=8) (actual time=65.421..65.422 rows=1.00
loops=1)
  Buffers: shared hit=152 read=1
  ->  Index Only Scan using idx_flows_attack_type_id on flows  (cost=0.43..3649.74
rows=171056 width=0)
        (actual time=1.202..38.636 rows=173559.00 loops=1)
        Index Cond: (attack_type_id = 2)
Execution Time: 65.799 ms  |  Total Buffer Pages Traversed: 153 pages (1.2 MB)
```

```
-- 2. WITHOUT INDEX (Parallel Sequential Table Scan across 2 workers)
Finalize Aggregate  (cost=70492.94..70492.95 rows=1 width=8) (actual time=693.690..702.394
rows=1.00 loops=1)
  Buffers: shared hit=1034 read=54658
  ->  Gather  (cost=70492.72..70492.93 rows=2 width=8) (actual time=693.461..702.383
rows=3.00 loops=1)
        Workers Planned: 2  |  Workers Launched: 2
        ->  Parallel Seq Scan on flows  (cost=0.00..69314.54 rows=71273 width=0)
              Filter: (attack_type_id = 2)  |  Rows Removed by Filter: 814236
Execution Time: 702.537 ms  |  Total Buffer Pages Traversed: 55,692 pages (435.1 MB)
```

■ EMPIRICAL INDEX OPTIMIZATION RESULTS

• Query Latency Reduction: Dropped from 702.5 ms (Sequential Scan) to 65.8 ms (Index Scan) -> 10.7x Speedup.

• I/O Buffer Traversal Reduction: Dropped from 55,692 disk pages (435 MB) to 153 index pages (1.2 MB) -> 364x I/O Reduction.

• Production Impact: Guarantees interactive SOC dashboard performance without saturating database server I/O.

8. Set-up and Execution Instructions

This section provides numbered, copy-paste-ready instructions to deploy and execute the pipeline from a clean development environment.

Step 1: System Prerequisites

Verify that the host environment meets the minimum software requirements:

- • **Operating System:** Windows 10/11, macOS 12+, or Ubuntu Linux 22.04 LTS.
- • **Python:** Python 3.10, 3.11, or 3.12 (64-bit). Verify with: python --version.
- • **PostgreSQL:** PostgreSQL 14, 15, 16, or 17 server running locally or accessible via network.
- • **System Memory & Disk:** Minimum 8 GB RAM and 10 GB free disk space (for dataset extraction and PostgreSQL tables).

Step 2: Clone the Repository and Navigate to Root

```
git clone https://github.com/NardosShumete/ids-etl-pipeline-postgres-assignment.git
```

```
cd ids-etl-pipeline-postgres-assignment
```

Step 3: Create and Activate Python Virtual Environment

```
# Windows PowerShell:
python -m venv .venv
.\.venv\Scripts\Activate.ps1

# Linux / macOS Bash:
python3 -m venv .venv
source .venv/bin/activate
```

Step 4: Install Required Python Packages

```
pip install --upgrade pip
pip install -r requirements.txt
# Verify key packages:
python -c 'import pandas, psycopg2, sqlalchemy, docx, matplotlib; print("Environment
Ready!")'
```

Step 5: Configure Environment Credentials (.env)

Copy the template configuration file .env.example to .env and configure your local PostgreSQL database credentials:

```
# Windows PowerShell:
Copy-Item .env.example .env

# Linux / macOS:
cp .env.example .env

# Edit .env with your PostgreSQL credentials:
DB_HOST=localhost
DB_PORT=5432
DB_NAME=ids_flows_db
DB_USER=postgres
DB_PASSWORD=your_secure_password_here
```

Step 6: Place the Dataset Archive

Download archive.zip from Kaggle (chethuhn/network-intrusion-dataset) and place it directly into the data/ directory:

```
# Verify archive placement (size should be ~229 MB):
ls -lh data/archive.zip
```

Step 7: Initialize PostgreSQL Database and Schema DDL

```
python -m src.create_database

# Expected Output:
# [INFO] Database 'ids_flows_db' created or verified.
# [INFO] Executing schema DDL from sql/01_schema.sql...
# [INFO] Tables created: source_files, attack_types, flows, flow_statistics.
# [INFO] Attack types seed taxonomy inserted (15 classes).
```

Step 8: Execute Full ETL Ingestion Pipeline

```
python -m src.pipeline

# Expected Output:
# [INFO] pipeline: Starting pipeline execution...
# [INFO] extract: Extracting data/archive.zip to data/raw/...
# [INFO] pipeline: Ingesting file 1/8: Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv...
# ...
# [INFO] pipeline: Ingestion completed: 2,616,266 records in 19m 35s.
# [INFO] pipeline: Applying post-load indexes from sql/02_indexes.sql...
# [INFO] pipeline: All B-Tree indexes created successfully.
```

Step 9: Run Automated Verification and Quality Audits

```
python -m src.verify

# Expected Output:
# [PASS] Dataset verification confirmed: > 2 Million rows ingested.
# [PASS] Vertical partitioning is 100% synchronized (flows == flow_statistics).
# [PASS] Zero orphan records found. Referential integrity is strictly maintained.
# [PASS] Zero NULL values in critical telemetry columns.
# [PASS] All attack type labels are clean, valid, and properly encoded.
```

Step 10: Run Unit Tests and Code Linter

```
pytest tests/ -v
ruff check src/ tests/

# Expected Output:
# ============================== 13 passed in 2.61s ==============================
# All checks passed!
```

8.1 Operational Troubleshooting Guide

Table 14 documents common deployment errors, root causes, and verified technical remediations:

Table 14: Operational Troubleshooting Guide

Observed Symptom / Error	Probable Root Cause	Verified Technical Remediation
ValueError: Missing required environment variable: DB_PASSWORD	.env file missing or DB_PASSWORD is unset.	Copy .env.example to .env and set your active PostgreSQL password.
psycopg2.OperationalError: Connection refused (port 5432)	PostgreSQL service is stopped or port is blocked.	Start service via 'net start postgresql-x64-16' (Windows) or 'sudo systemctl start postgresql' (Linux).
FileNotFoundError: archive.zip not found in data/	Dataset archive was not placed in data/ folder.	Download archive.zip from Kaggle and place it in data/archive.zip.
PermissionError: [Errno 13] Permission denied: 'docs/report.docx'	Report file is currently open in Microsoft Word.	Close Microsoft Word to release the exclusive file lock and re-run build_report.py.
MemoryError during CSV extraction	chunk_size set too high for available system RAM.	Ensure DEFAULT_CHUNK_SIZE remains set to 200,000 in src/config.py.

9. Challenges and Lessons Learned

Developing an enterprise-grade ETL pipeline across millions of records yielded several critical engineering lessons:

- **1. Real-World Datasets Contain Hidden Header Defects:** Raw network data invariably contains irregular whitespace and duplicate column names ('Fwd Header Length'). Standardizing column names into clean, predictable snake_case identifiers early in the pipeline prevents downstream relational errors.
- **2. Character Encodings Require Fallback Resilience:** Assuming a uniform UTF-8 encoding across all files causes unexpected fatal crashes. Implementing automated dual-encoding fallback (UTF-8 with Latin-1 fallback) guarantees robust extraction across legacy and multi-platform data dumps.
- **3. Chunked Streaming is Mandatory for Predictable Memory:** Attempting full-dataset in-memory processing exhausts RAM. Streaming chunk generators bound memory predictably (643.6 MB peak RAM), allowing the pipeline to execute reliably on constrained developer machines.
- **4. Synchronizing 1:1 Vertical Partitions Requires Explicit Sequence Pre-allocation:** When splitting a single DataFrame into two relational tables with a 1:1 foreign key dependency, relying on auto-incrementing database defaults can cause sequence desynchronization during parallel or aborted transactions. Explicitly pre-allocating sequences via nextval('flows_id_seq') guarantees perfect 1:1 ID alignment.
- **5. Bulk Ingestion Must Defer Index Creation:** Inserting rows into indexed tables triggers severe B-Tree page splits and Write-Ahead Log (WAL) overhead. Deferring secondary index construction until after bulk loading reduced total load time by an estimated 65%.

10. Limitations and Future Work

While this architecture successfully meets all technical objectives, several architectural enhancements can expand its capabilities:

- • **Declarative Table Partitioning by Range:** Currently, flows is a single large table. Implementing PostgreSQL declarative table partitioning by capture date (PARTITION BY RANGE (imported_at)) would allow partition pruning during historical range queries.
- • **Distributed Workflow Orchestration (Apache Airflow):** Migrating from local Python script execution to an Apache Airflow DAG would provide automated scheduling, DAG visualization, alerting, and automated retries.
- • **Real-Time Stream Ingestion (Apache Kafka):** Integrating Apache Kafka and Faust would transition the system from batch file ingestion to real-time packet stream processing, allowing sub-second intrusion alerts.
- • **Interactive SOC Dashboards (Power BI / Grafana):** Connecting PostgreSQL to Grafana or Microsoft Power BI would provide operational security dashboards for real-time visualization of PortScan and DoS spikes.
- • **Machine Learning Intrusion Classification:** Deploying an XGBoost or Random Forest classification model directly on the normalized flow_statistics telemetry would enable automated anomaly detection for zero-day exploits.

11. Team Contributions

This project was collaboratively designed and implemented by a multidisciplinary team of student data engineers. Table 15 specifies the technical responsibilities and contributions of each team member:

Table 15: Comprehensive Team Contributions Matrix

Team Member Name	Student ID	Engineering Role	Detailed Technical Deliverables
[TO BE FILLED: Student 1]	[TO BE FILLED: ID 1]	Pipeline Architecture & Streaming Ingestion Lead	Designed src/extract.py, streaming generator architecture, chunksize optimization, and end-to-end pipeline CLI orchestration.
[TO BE FILLED: Student 2]	[TO BE FILLED: ID 2]	Relational Database Modeling Specialist	Authored sql/01_schema.sql, engineered 1:1 vertical table partitioning, primary/foreign key constraints, and PostgreSQL data types.
[TO BE FILLED: Student 3]	[TO BE FILLED: ID 3]	Data Quality & Cleansing Engineer	Developed src/transform.py, regex Web Attack encoding sanitization, infinite/NaN filtering, negative duration purging, and unit tests.

Team Member Name	Student ID	Engineering Role	Detailed Technical Deliverables
[TO BE FILLED: Student 4]	[TO BE FILLED: ID 4]	Analytical Query & Optimization Specialist	Authored sql/02_indexes.sql, sql/04_analytics.sql, post-load indexing benchmarks, and EXPLAIN ANALYZE query optimization.

12. Conclusion

This project successfully engineered and demonstrated a resilient, production-grade ETL pipeline and normalized relational data warehouse for network intrusion detection telemetry. By processing the benchmark CIC-IDS-2017 dataset, the architecture extracted 2,830,743 raw records across 8 capture files, purged 214,477 invalid records through deterministic cleaning rules, and successfully ingested exactly 2,616,266 records into PostgreSQL with zero referential integrity violations.

Key engineering achievements include streaming binary COPY loading at 2,226 rows per second, strict memory bounding below 650 MB RAM, a 3.76x cache density improvement through 1:1 vertical table partitioning, and a 10.7x analytical query speedup via strategic post-load B-Tree indexing. The resulting system empowers security operations analysts to conduct rapid, forensic threat hunting across millions of network connections, providing a robust technical foundation for modern cybersecurity data engineering.

13. References

- **1. Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018).** Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP 2018), pp. 108-116.
- **2. Canadian Institute for Cybersecurity (CIC).** CIC-IDS-2017 Dataset Documentation and Feature Definitions. University of New Brunswick (UNB). https://www.unb.ca/cic/datasets/ids-2017.html
- **3. PostgreSQL Global Development Group. (2024).** PostgreSQL 16 Documentation: Routine Vacuuming, Indexing Strategies, and COPY Protocol. https://www.postgresql.org/docs/16/
- **4. McKinney, W. (2022).** Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter (3rd ed.). O'Reilly Media.
- **5. Kaggle Dataset Mirror.** Network Intrusion Dataset (CIC-IDS-2017). Curated by Chethuhn. https://www.kaggle.com/datasets/chethuhn/network-intrusion-dataset

Appendix A: Project Repository and Source Code Link

The complete, version-controlled source code, unit test suite, SQL DDL scripts, and report generators are hosted on GitHub:

Repository: GitHub Repository URL: https://github.com/NardosShumete/ids-etl-pipeline-postgres-assignment.git

Appendix B: Project Folder Layout

```
ids-etl-pipeline-postgres-assignment/
├── .env.example              <- Template configuration file for database credentials
├── .gitignore                <- Strict gitignore excluding datasets, logs, and environments
├── README.md                 <- Complete setup, usage, and architectural documentation
├── requirements.txt          <- Pinned Python dependencies (pandas, psycopg2, docx, etc.)
├── data/
│   ├── archive.zip           <- Raw Kaggle CIC-IDS-2017 compressed dataset (229 MB)
│   └── raw/                  <- Extracted CSV files (843.66 MB total across 8 files)
├── docs/
│   ├── ETL_Pipeline_Report.docx <- Comprehensive formal Word technical report
│   ├── ETL_Pipeline_Report.pdf  <- High-fidelity PDF export of the technical report
│   ├── build_report.py       <- Automated report generator script
│   └── figures/              <- Publication-grade figures (300 DPI PNG diagrams and charts)
│       ├── architecture_diagram.png
│       ├── er_diagram.png
│       ├── attack_distribution.png
│       ├── top_attack_ports.png
│       ├── daily_traffic_volume.png
│       └── index_performance_comparison.png
├── logs/
│   └── etl.log               <- Execution log tracing chunk ingestion and timestamps
├── sql/
│   ├── 01_schema.sql         <- PostgreSQL DDL schema with 1:1 vertical partitioning
│   ├── 02_indexes.sql        <- Post-load B-Tree indexes and performance rationale
│   ├── 03_verification.sql   <- Comprehensive database integrity and quality queries
│   └── 04_analytics.sql      <- SOC intrusion threat-hunting analytical SQL queries
├── src/
│   ├── __init__.py           <- Package initialization
│   ├── config.py             <- Path management and strict credential validation
│   ├── create_database.py    <- Automated database creation and schema executor
│   ├── extract.py            <- Streaming CSV reader with dual-encoding fallback
│   ├── transform.py          <- Data cleansing rules, label repair, and vertical split
│   ├── load.py               <- High-throughput streaming COPY loader (psycopg2)
│   ├── logger.py             <- Dual console and file logging subsystem
│   ├── pipeline.py           <- End-to-end pipeline orchestrator and CLI entry point
│   └── verify.py             <- Automated database verification audit runner
└── tests/
    ├── test_config.py        <- Unit tests for configuration and password validation
    ├── test_schema.py        <- Integration tests for tables, PK/FKs, and check constraints
    └── test_transform.py     <- Unit tests for cleansing, deduplication, and partitioning
```

Appendix C: Complete PostgreSQL Schema DDL

```
-- ================================================================================
-- CIC-IDS-2017 Intrusion Detection System ETL Pipeline: Database Schema
-- Target: PostgreSQL 14+
-- Scenario: Network Operations / SOC Analytics for ISP / Financial Infrastructure
-- ================================================================================

-- Drop existing tables in reverse dependency order to ensure idempotency
DROP TABLE IF EXISTS flow_statistics CASCADE;
DROP TABLE IF EXISTS flows CASCADE;
DROP TABLE IF EXISTS source_files CASCADE;
DROP TABLE IF EXISTS attack_types CASCADE;

-- --------------------------------------------------------------------------------
-- 1. attack_types (Lookup / Dimension Table)
-- Normalizes categorical attack taxonomy (15+ classes), decoupling text labels
-- from high-volume flow rows to optimize storage and enforce 3NF.
-- --------------------------------------------------------------------------------
CREATE TABLE attack_types (
    id SMALLSERIAL PRIMARY KEY,
    label VARCHAR(64) NOT NULL UNIQUE,
    category VARCHAR(32) NOT NULL,
    is_malicious BOOLEAN NOT NULL DEFAULT FALSE,
    description TEXT
);

COMMENT ON TABLE attack_types IS 'Lookup dimension mapping network flow labels to
standardized threat categories';
COMMENT ON COLUMN attack_types.label IS 'Exact string label as reported by IDS or flow
generator (e.g., BENIGN, DoS Hulk)';
COMMENT ON COLUMN attack_types.category IS 'High-level threat classification (e.g., Benign,
DoS, DDoS, PortScan, Brute-Force, Web Attack, Bot, Infiltration, Heartbleed)';
COMMENT ON COLUMN attack_types.is_malicious IS 'Boolean flag indicating whether the flow
represents malicious adversary activity';

-- --------------------------------------------------------------------------------
-- 2. source_files (ETL Audit and Lineage Table)
-- Tracks source CSV file metadata, ingest lineage, and record counts for auditability.
-- --------------------------------------------------------------------------------
CREATE TABLE source_files (
    id SERIAL PRIMARY KEY,
    file_name VARCHAR(255) NOT NULL UNIQUE,
    capture_day VARCHAR(32),
    row_count BIGINT DEFAULT 0 CHECK (row_count >= 0),
    imported_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

COMMENT ON TABLE source_files IS 'Tracks source dataset files, ingestion timestamps, and raw
row counts for lineage';
COMMENT ON COLUMN source_files.file_name IS 'Base filename of the raw CSV file extracted
from CIC-IDS-2017';
COMMENT ON COLUMN source_files.capture_day IS 'Weekday of packet capture during dataset
generation (e.g., Monday, Tuesday)';

-- --------------------------------------------------------------------------------
-- 3. flows (Core Transactional Telemetry Table)
-- Stores primary network flow indicators, connection endpoints, duration,
-- byte/packet totals, rates, and primary TCP control flags.
-- Designed for high-frequency filtering, SOC dashboard queries, and rule evaluation.
```

```
-- ------------------------------------------------------------------------
CREATE TABLE flows (
    id BIGSERIAL PRIMARY KEY,
    source_file_id INT NOT NULL REFERENCES source_files(id) ON DELETE CASCADE,
    attack_type_id SMALLINT NOT NULL REFERENCES attack_types(id) ON DELETE RESTRICT,
    destination_port INTEGER NOT NULL CHECK (destination_port >= 0 AND destination_port <=
65535),
    flow_duration BIGINT NOT NULL CHECK (flow_duration >= 0),
    total_fwd_packets BIGINT NOT NULL CHECK (total_fwd_packets >= 0),
    total_bwd_packets BIGINT NOT NULL CHECK (total_bwd_packets >= 0),
    total_length_of_fwd_packets BIGINT NOT NULL CHECK (total_length_of_fwd_packets >= 0),
    total_length_of_bwd_packets BIGINT NOT NULL CHECK (total_length_of_bwd_packets >= 0),
    flow_bytes_per_s DOUBLE PRECISION,
    flow_packets_per_s DOUBLE PRECISION,
    flow_iat_mean DOUBLE PRECISION,
    flow_iat_std DOUBLE PRECISION,
    flow_iat_max DOUBLE PRECISION,
    flow_iat_min DOUBLE PRECISION,
    fin_flag_count SMALLINT NOT NULL DEFAULT 0 CHECK (fin_flag_count >= 0),
    syn_flag_count SMALLINT NOT NULL DEFAULT 0 CHECK (syn_flag_count >= 0),
    rst_flag_count SMALLINT NOT NULL DEFAULT 0 CHECK (rst_flag_count >= 0),
    psh_flag_count SMALLINT NOT NULL DEFAULT 0 CHECK (psh_flag_count >= 0),
    ack_flag_count SMALLINT NOT NULL DEFAULT 0 CHECK (ack_flag_count >= 0),
    urg_flag_count SMALLINT NOT NULL DEFAULT 0 CHECK (urg_flag_count >= 0),
    cwe_flag_count SMALLINT NOT NULL DEFAULT 0 CHECK (cwe_flag_count >= 0),
    ece_flag_count SMALLINT NOT NULL DEFAULT 0 CHECK (ece_flag_count >= 0),
    down_up_ratio DOUBLE PRECISION CHECK (down_up_ratio >= 0),
    average_packet_size DOUBLE PRECISION CHECK (average_packet_size >= 0)
);

COMMENT ON TABLE flows IS 'Primary network flow telemetry records containing connection
endpoints and core flow volume';
COMMENT ON COLUMN flows.destination_port IS 'Destination TCP/UDP service port (0-65535)';
COMMENT ON COLUMN flows.flow_duration IS 'Total duration of the bidirectional flow in
microseconds';
COMMENT ON COLUMN flows.total_fwd_packets IS 'Total number of packets transmitted in the
forward direction (source to dest)';
COMMENT ON COLUMN flows.total_bwd_packets IS 'Total number of packets transmitted in the
backward direction (dest to source)';

-- ------------------------------------------------------------------------
-- 4. flow_statistics (Detailed Flow Metrics Table - 1:1 Partition)
-- Vertically partitioned from the core `flows` table to maintain compact row
-- widths in PostgreSQL buffer cache pages. Holds deep packet length distributions,
-- directional IATs, TCP window metrics, subflow stats, and active/idle state timing.
-- ------------------------------------------------------------------------
CREATE TABLE flow_statistics (
    flow_id BIGINT PRIMARY KEY REFERENCES flows(id) ON DELETE CASCADE,

    -- Packet Length Distributions
    fwd_packet_length_max DOUBLE PRECISION CHECK (fwd_packet_length_max >= 0),
    fwd_packet_length_min DOUBLE PRECISION CHECK (fwd_packet_length_min >= 0),
    fwd_packet_length_mean DOUBLE PRECISION CHECK (fwd_packet_length_mean >= 0),
    fwd_packet_length_std DOUBLE PRECISION CHECK (fwd_packet_length_std >= 0),
    bwd_packet_length_max DOUBLE PRECISION CHECK (bwd_packet_length_max >= 0),
    bwd_packet_length_min DOUBLE PRECISION CHECK (bwd_packet_length_min >= 0),
    bwd_packet_length_mean DOUBLE PRECISION CHECK (bwd_packet_length_mean >= 0),
    bwd_packet_length_std DOUBLE PRECISION CHECK (bwd_packet_length_std >= 0),
    min_packet_length DOUBLE PRECISION CHECK (min_packet_length >= 0),
    max_packet_length DOUBLE PRECISION CHECK (max_packet_length >= 0),
```

```
    packet_length_mean DOUBLE PRECISION CHECK (packet_length_mean >= 0),
    packet_length_std DOUBLE PRECISION CHECK (packet_length_std >= 0),
    packet_length_variance DOUBLE PRECISION CHECK (packet_length_variance >= 0),

    -- Directional Inter-Arrival Times (IAT)
    fwd_iat_total DOUBLE PRECISION CHECK (fwd_iat_total >= 0),
    fwd_iat_mean DOUBLE PRECISION CHECK (fwd_iat_mean >= 0),
    fwd_iat_std DOUBLE PRECISION CHECK (fwd_iat_std >= 0),
    fwd_iat_max DOUBLE PRECISION CHECK (fwd_iat_max >= 0),
    fwd_iat_min DOUBLE PRECISION CHECK (fwd_iat_min >= 0),
    bwd_iat_total DOUBLE PRECISION CHECK (bwd_iat_total >= 0),
    bwd_iat_mean DOUBLE PRECISION CHECK (bwd_iat_mean >= 0),
    bwd_iat_std DOUBLE PRECISION CHECK (bwd_iat_std >= 0),
    bwd_iat_max DOUBLE PRECISION CHECK (bwd_iat_max >= 0),
    bwd_iat_min DOUBLE PRECISION CHECK (bwd_iat_min >= 0),

    -- Segment, Header & Rate Metrics
    fwd_psh_flags SMALLINT CHECK (fwd_psh_flags >= 0),
    bwd_psh_flags SMALLINT CHECK (bwd_psh_flags >= 0),
    fwd_urg_flags SMALLINT CHECK (fwd_urg_flags >= 0),
    bwd_urg_flags SMALLINT CHECK (bwd_urg_flags >= 0),
    fwd_header_length BIGINT,
    bwd_header_length BIGINT,
    fwd_packets_per_s DOUBLE PRECISION,
    bwd_packets_per_s DOUBLE PRECISION,
    avg_fwd_segment_size DOUBLE PRECISION CHECK (avg_fwd_segment_size >= 0),
    avg_bwd_segment_size DOUBLE PRECISION CHECK (avg_bwd_segment_size >= 0),
    fwd_header_length_1 BIGINT,

    -- Bulk Transfer Metrics
    fwd_avg_bytes_bulk DOUBLE PRECISION CHECK (fwd_avg_bytes_bulk >= 0),
    fwd_avg_packets_bulk DOUBLE PRECISION CHECK (fwd_avg_packets_bulk >= 0),
    fwd_avg_bulk_rate DOUBLE PRECISION CHECK (fwd_avg_bulk_rate >= 0),
    bwd_avg_bytes_bulk DOUBLE PRECISION CHECK (bwd_avg_bytes_bulk >= 0),
    bwd_avg_packets_bulk DOUBLE PRECISION CHECK (bwd_avg_packets_bulk >= 0),
    bwd_avg_bulk_rate DOUBLE PRECISION CHECK (bwd_avg_bulk_rate >= 0),

    -- Subflows and TCP Window Metrics
    subflow_fwd_packets BIGINT CHECK (subflow_fwd_packets >= 0),
    subflow_fwd_bytes BIGINT CHECK (subflow_fwd_bytes >= 0),
    subflow_bwd_packets BIGINT CHECK (subflow_bwd_packets >= 0),
    subflow_bwd_bytes BIGINT CHECK (subflow_bwd_bytes >= 0),
    init_win_bytes_forward BIGINT,
    init_win_bytes_backward BIGINT,
    act_data_pkt_fwd BIGINT CHECK (act_data_pkt_fwd >= 0),
    min_seg_size_forward BIGINT,

    -- Active / Idle Connection Timing
    active_mean DOUBLE PRECISION CHECK (active_mean >= 0),
    active_std DOUBLE PRECISION CHECK (active_std >= 0),
    active_max DOUBLE PRECISION CHECK (active_max >= 0),
    active_min DOUBLE PRECISION CHECK (active_min >= 0),
    idle_mean DOUBLE PRECISION CHECK (idle_mean >= 0),
    idle_std DOUBLE PRECISION CHECK (idle_std >= 0),
    idle_max DOUBLE PRECISION CHECK (idle_max >= 0),
    idle_min DOUBLE PRECISION CHECK (idle_min >= 0)
);

COMMENT ON TABLE flow_statistics IS 'Secondary 1:1 table storing granular statistical
distributions, subflow aggregates, and timing';
```

```
-- ------------------------------------------------------------------------------
-- 5. Seed attack_types Lookup Table
-- Populates all known attack labels across the CIC-IDS-2017 dataset
-- ------------------------------------------------------------------------------
INSERT INTO attack_types (label, category, is_malicious, description) VALUES
('BENIGN', 'Benign', FALSE, 'Normal, authorized background network traffic across campus and
enterprise workstations'),
('DoS Hulk', 'DoS', TRUE, 'HTTP Unbearable Load King: High-volume application-layer DoS
flooding with dynamic randomized request URLs'),
('PortScan', 'PortScan', TRUE, 'Probing host TCP/UDP ports to discover open listening
services and perimeter vulnerabilities'),
('DDoS', 'DDoS', TRUE, 'Distributed Denial of Service attack overwhelming target network
bandwidth and socket buffers'),
('DoS GoldenEye', 'DoS', TRUE, 'Layer 7 DoS attack issuing persistent keep-alive HTTP
GET/POST requests with cache-busting headers'),
('FTP-Patator', 'Brute-Force', TRUE, 'Multi-threaded brute-force password guessing targeting
File Transfer Protocol (FTP) servers'),
('SSH-Patator', 'Brute-Force', TRUE, 'Multi-threaded brute-force password guessing targeting
Secure Shell (SSH) daemons'),
('DoS slowloris', 'DoS', TRUE, 'Low-bandwidth Layer 7 attack keeping hundreds of HTTP
connections open by slowly transmitting incomplete headers'),
('DoS Slowhttptest', 'DoS', TRUE, 'Slow HTTP POST / Slow Read attack exhausting web server
connection pools and memory'),
('Bot', 'Bot', TRUE, 'Compromised workstation participating in Ares botnet command and
control (C2) communication'),
('Web Attack - Brute Force', 'Web Attack', TRUE, 'Automated dictionary and credential
stuffing attack against web application authentication forms'),
('Web Attack - XSS', 'Web Attack', TRUE, 'Cross-Site Scripting attack injecting malicious
client-side JavaScript payloads into web forms'),
('Web Attack - Sql Injection', 'Web Attack', TRUE, 'SQL injection exploitation attempting
unauthorized relational database access, bypass, or exfiltration'),
('Infiltration', 'Infiltration', TRUE, 'Post-exploitation lateral movement and malware
payload staging across internal network subnets'),
('Heartbleed', 'Heartbleed', TRUE, 'Exploitation of OpenSSL TLS heartbeat extension buffer
over-read (CVE-2014-0160) to leak private keys and memory')
ON CONFLICT (label) DO UPDATE
SET category = EXCLUDED.category,
    is_malicious = EXCLUDED.is_malicious,
    description = EXCLUDED.description;
```